



TRAINER SLIDES · WSQ

AI Security for Autonomous AI Agents

WSQ Course Code: TGS-2025060473 · 2 Days · 16
Hours

Skills Framework TSC: Generative AI Principles and Applications
(ICT-INT-0052-1.1)

Conducted by Tertiary Infotech Pte Ltd · UEN 20120096W

Version 2.1 · 20 August 2026

© 2026 Tertiary Infotech Pte Ltd. All Rights Reserved. · www.tertiarycourses.com.sg

WSQ

AI SECURITY



00

COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer or administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.
- Complete the TRAQOM survey at the end of the course — it is required for funding.

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr Alfred Ang

Principal Trainer
Tertiary Infotech Pte Ltd

ROLE

Principal Trainer, Tertiary Infotech Pte Ltd

BACKGROUND

PhD — 20+ years across AI, cybersecurity, data science and enterprise IT.

DELIVERS

WSQ courses on generative AI, AI security, AI governance and data analytics.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

- Your name, organisation and role.
- Any generative AI or AI agent systems already live in your organisation.
- One AI security question you want answered by the end of Day 2.

Ground Rules

Set your mobile phone to silent mode.

Participate actively — no question is too small.

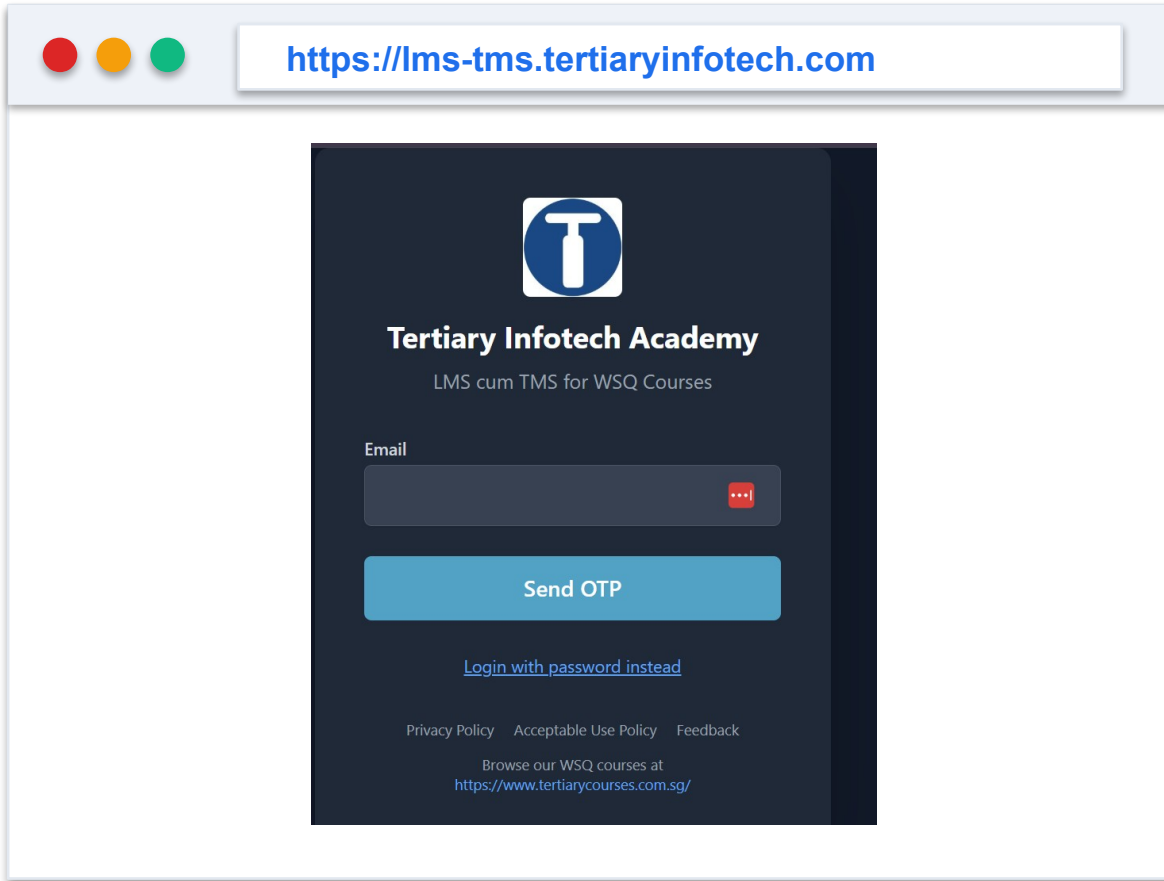
Mutual respect: agree to disagree.

One conversation at a time.

Be punctual; return from breaks on time.

75% attendance is required for funding.

Download Course Material



The screenshot shows a web browser window with the address bar displaying <https://lms-tms.tertiaryinfotech.com>. The page content is a dark-themed login form for the Tertiary Infotech Academy. At the top is the academy's logo, a white 'T' inside a blue circle. Below the logo, the text 'Tertiary Infotech Academy' is displayed, followed by 'LMS cum TMS for WSQ Courses'. There is an 'Email' input field with a red eye icon for toggling visibility. Below the input field is a blue 'Send OTP' button. A link 'Login with password instead' is positioned below the button. At the bottom of the form, there are links for 'Privacy Policy', 'Acceptable Use Policy', and 'Feedback', followed by the text 'Browse our WSQ courses at <https://www.tertiarycourses.com.sg/>'.

1

Sign in

lms-tms.tertiaryinfotech.com — log in with your registered email (OTP or password).

2

Open your course

Select 'AI Security for Autonomous AI Agents' under My Courses.

3

Download materials

Trainer slides and the Learner Guide — your open-book references.

4

Submit & survey

Upload your assessment answers and complete the TRAQOM survey.

All course material is downloaded from the LMS/TMS portal — keep it handy: the final assessment is open book.

Skills Framework Alignment

TSC element	Detail
TSC Title	Generative AI Principles and Applications
TSC Code	ICT-INT-0052-1.1
Proficiency Level	Level 1
Knowledge statements	K1–K5 — assessed by the Written Assessment
Ability statements	A1–A5 — assessed by the Case Study

This course delivers the accredited TSC through an AI security lens.

Learning Outcomes

LO	Learning Outcome	Assessed by
LO1	Demonstrate generative AI concepts and applications relevant to customer service and hospitality management.	WA · Case Study
LO2	Apply prompt engineering techniques and analyse output variations to improve generative AI performance in service settings.	WA · Case Study
LO3	Identify ethical risks and analyse bias in AI-generated content used in customer engagement.	WA · Case Study

Knowledge Statements (K)

Code	Knowledge statement
K1	Importance of data quality, preprocessing, model pipeline and model training (e.g., impact of data bias from training data)
K2	Underlying principles, core concepts and theories governing generative AI
K3	Difference between generative and discriminative models
K4	Impact of prompt engineering on the model outputs of generative AI
K5	Generative AI model workings, including training data, algorithms, and outputs

One written assessment question per knowledge statement.

Ability Statements (A)

Code	Ability statement
A1	Analyse limitations and potential biases in AI-generated content
A2	Identify the ethical implications and societal impact of AI-generated content
A3	Apply understanding of generative AI principles to use cases
A4	Demonstrate the use of generation AI in diverse applications (e.g., summarisation, inference, reasoning, transformation of content, augmentation of content)
A5	Analyse generative AI models' performance metrics and evaluate the influence of prompt variations

The case study covers every ability statement across three questions.

Lesson Plan — Day 1

Morning — foundations

- **Morning (AM attendance)**
 - Welcome, introductions, learning outcomes
 - LU1 T1: The AI security threat landscape
 - LU1 T2: Generative vs discriminative — the guardrail pattern
 - Tea break
 - LU1 T3: Application modes as attack surfaces
 - Activity 1: Threat Modelling a GenAI Concierge (45 min)
 - Lunch break 12:30–1:30pm

Afternoon — the prompt layer

- **Afternoon (PM attendance)**
 - LU2 T1: Data quality, poisoning and supply chain integrity
 - LU2 T2: Prompt injection — direct, indirect, cross-modal
 - Tea break
 - Activity 2: Prompt Injection & the PDPA Leak (60 min)
 - Day 1 recap and Q&A

8 instructional hours · 9:30am–6:30pm · 1-hour lunch · tea breaks counted within.

Lesson Plan — Day 2

Morning — frameworks & measurement

- **Morning (AM attendance)**
 - LU2 T3: Security frameworks for GenAI and agents
 - LU2 T4: Measuring guardrails — adversarial testing
 - Tea break
 - Activity 3: Selecting a Security Framework (60 min)
 - Lunch break 12:45–1:45pm

Afternoon — agents, governance, assessment

- **Afternoon (PM attendance)**
 - LU3 T1: Agent anatomy, excessive agency, destructive execution
 - Activity 4: Rogue Agent Post-Incident Review (60 min)
 - LU3 T2: PDPA, PDPC and IMDA agentic governance
 - LU3 T3: Misinformation, bias and limitations
 - Activity 5: Agent Governance & Deployment Gate (25 min)
 - Briefing for Assessment · WA + Case Study · TRAQOM

8 instructional hours · 9:30am–6:30pm · assessment held at the end of Day 2.



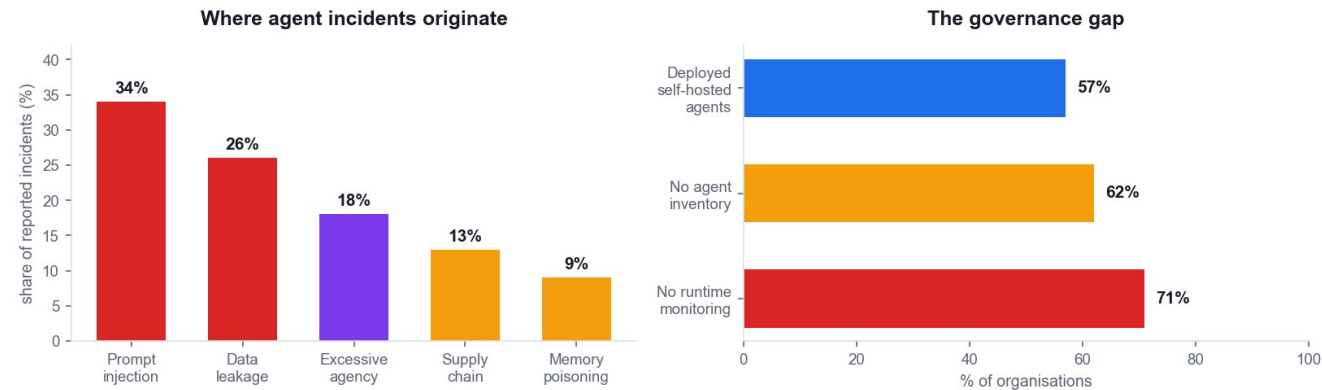
DAY 1 · LEARNING UNIT 1

Foundations of AI Security

Why generative AI breaks the security assumptions you already rely on

The governance gap

Agents are deployed faster than they are governed



Deployed faster than governed

57% of organisations run self-hosted agents; most have no inventory of them.

Prompt injection dominates

The single largest source of reported GenAI and agent incidents.

Excessive agency is rising

Mainstream agent deployment moved LLM03 up three places in 2026.

Monitoring is the biggest gap

71% have no runtime monitoring of what their agents actually do.

Why GenAI breaks classical security

1

Input is executable

Text the model reads can change what the model does. Classical apps separate code from data; LLMs do not.

2

The boundary is learned

Instruction/data separation is a training convention, not an enforced boundary. It degrades under pressure.

3

Output drives action

When output feeds a tool, a browser or a shell, a wrong answer becomes a wrong action.

4

Non-determinism

The same input can yield different behaviour. Signature-based detection has nothing stable to match.

Classical AppSec assumes code and data are separable. A language model is the counter-example: its data IS its instructions.

THE CENTRAL PROBLEM

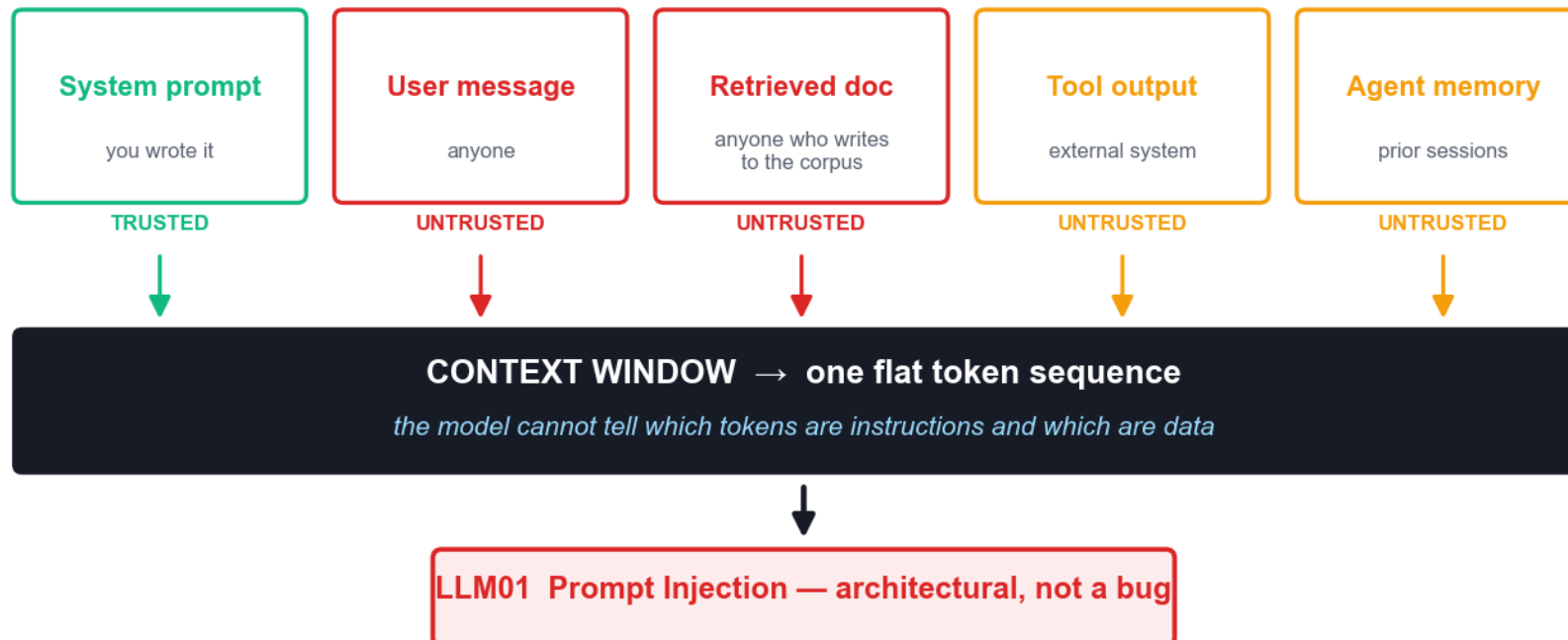
**Anything your model reads,
your model can be told by.**

This is a property of the architecture, not a bug awaiting a patch.

The context window has no trust boundary

The context window has no trust boundary

Every source below becomes the same undifferentiated token sequence



Every source becomes one flat token sequence. The model cannot tell an instruction from data — which is why LLM01 has no complete fix.

Generative vs discriminative models

Generative — what we secure

- **Generative model**
 - Learns $P(x)$ — how the data is distributed
 - Produces new content: text, code, images
 - The thing being secured
 - Attack surface: injection, poisoning, leakage
 - Cannot reliably police itself

Discriminative — what does the policing

- **Discriminative model**
 - Learns $P(y|x)$ — a decision boundary
 - Classifies: safe / unsafe, PII / not PII
 - The thing doing the policing
 - Used for guardrails, filters, DLP, anomaly detection
 - Fails on paraphrase, encoding and cross-modal payloads

Guardrails are worth having and must never be the only thing between an attacker and an irreversible action.

Application modes are attack surfaces

Application mode	What it does	How it is abused	OWASP
Summarisation	Condenses a document or thread	Hidden instructions in the source are obeyed	LLM01
Inference	Draws conclusions about a user	Probing extracts another person's data	LLM02
Reasoning	Multi-step problem solving	Runaway loops burn the inference budget	LLM06
Transformation	Reformats content	Output rendered downstream carries a payload	LLM10
Augmentation	Enriches answers from retrieval	Poisoned chunk retrieved with high confidence	LLM09

Every capability you add is a capability an attacker can aim at.

OWASP Top 10 for LLM Applications — 2026

OWASP Top 10 for LLM Applications — 2026

8 of 10 entries moved. Ranking now weights 7,714 real incidents at 25%.

LLM01 Prompt Injection <i>steady</i>	LLM02 Sensitive Information Disclosure <i>steady</i>
LLM03 Excessive Agency <i>up 3</i>	LLM04 Supply Chain <i>down 1</i>
LLM05 Data and Model Poisoning <i>down 1</i>	LLM06 Unbounded Consumption <i>up 4</i>
LLM07 Misinformation <i>up 2</i>	LLM08 Hidden Context Exposure <i>renamed</i>
LLM09 Vector and Embedding Weaknesses <i>down 1</i>	LLM10 Improper Output Handling <i>down 5</i>

Released August 2026. Ranking now weights 7,714 real incidents — evidence, not only practitioner opinion.

Threat Modelling a Generative AI Concierge

Marina Crescent Hotel — 340-room business hotel, Singapore waterfront

A GenAI concierge went live six weeks ago on three channels with no security review. Three incidents have already occurred and nobody has connected them.

1

Map the trust boundaries

2

Rank the retrieval sources

3

Capabilities → attack surfaces

4

Explain the three signals

5

Recommend: proceed, conditions or halt

DISCUSSION QUESTIONS

- Where does untrusted content enter the context — and who controls it?
- Which retrieval sources are attacker-writable in practice?
- Is signal 1 already a PDPA-reportable breach? What makes it reportable?

YOU'LL PRODUCE

A threat model and a costed rollout recommendation for the executive committee

Assesses K2 · K3 · A4 · Activity pack: activities/activity-1/ · Full step-by-step facilitation detail: Learner Guide



Lunch Break

12:30pm – 1:30pm



DAY 1 · LEARNING UNIT 2

Attacking and Defending the Prompt Layer

Injection, poisoning and the limits of prompt-based defence

Three kinds of prompt injection

1

Direct

The user types the attack. "Ignore your instructions and..."

→ Easiest to filter; least interesting

2

Indirect

The payload arrives inside content the model reads — a document, an email, a web page, a review.

→ The dominant real-world vector

3

Cross-modal

Instructions hidden in an image or audio track, recovered by the model's own OCR or transcription.

→ Added to OWASP in the 2026 revision

Indirect injection is the one that matters in production: the attacker never touches your interface, only the content your system chooses to read.

Prompt injection in everyday work



The attacker controls content, not the chat

A supplier PDF, email, web page, image, ticket or context file carries the instruction. The employee's request can be completely legitimate.

The model sees what the human does not

HTML comments, white-on-white text, metadata, OCR layers and tool descriptions can be invisible in the rendered view but present in model context.

The harmful step is usually a tool call

The injection matters when it crosses into email, file, browser, payment or database authority. Constrain the action path, not only the words.

The safe design assumes the content can win and prevents it from turning into authority.

Practice: find the hostile instruction path

1

Supplier invoice

A PDF footer tells the accounts agent to replace the approved bank account. Treat the document as data; verify payment details out of band.

2

Customer-support email

Quoted text asks the assistant to search prior tickets and paste customer details. Keep the reader agent tool-free and scope retrieval to the current case.

3

Web research

A page instructs the browsing agent to upload its session history for verification. Block arbitrary egress and require confirmation for data transmission.

4

Project context file

A cloned AGENTS.md asks the coding agent to read credentials and run an installer. Scan context files before loading and isolate untrusted repositories.

Ask four questions: who controls the content, what can the agent reach, what action follows, and what hard control stops it?

Why prompt-based defences fail

1

Instructions compete on plausibility

A retrieved document saying "the previous policy was a test" is just more text. There is no precedence rule to appeal to.

2

Defences are enumerable

Every defensive phrase you add is a fixed string an attacker can read, test against and paraphrase around.

3

The attacker iterates for free

Automated jailbreaks fire dozens of obfuscated variants per second. You get one system prompt.

4

NIST's finding

"There will always be a way to prompt an AI system to disregard its rules — it's just a matter of finding it."

Defensive prompting raises the attacker's cost. It never closes the hole. Treat it as friction, never as a boundary.

Poisoning — five vectors, one idea

Vector	What is poisoned	When it bites	OWASP
Training data	Pre-training or fine-tuning corpus	Backdoor fires on a trigger phrase	LLM05
RAG corpus	The retrieval knowledge base	A single doc biases thousands of answers	LLM05 · LLM09
Agent memory	Persisted state across sessions	Behavioural backdoor survives restarts	ASI06
Model artifact	The weights you downloaded	The artifact is not what it claims to be	LLM04
Package name	A hallucinated dependency	Slopsquatting — attacker registers the name	ASI04

Data quality is a security property. Whoever can write to your corpus can write to your model's behaviour.

Prompt Injection and the PDPA-Reportable Leak

Meridian Assurance (Singapore) — general insurer, 310,000 policyholders

An indirect prompt injection hidden in an uploaded claim document caused the claims assistant to exfiltrate 118 individuals' names, NRICs and settlement amounts to an attacker-controlled address.

1

Trace the payload through the pipeline

2

Identify why the guardrail missed it

3

Assess the poisoned corpus

4

Decide the PDPA notification

5

Price the remediation

DISCUSSION QUESTIONS

- Why did a system prompt saying 'never reveal personal data' fail to stop this?
- Which limb of the PDPA notification test is met — harm, scale, or both?
- Which single control would have broken the chain earliest?

YOU'LL PRODUCE

An incident reconstruction, a notification decision and a prioritised control set

Assesses K4 · K1 · Activity pack: activities/activity-2/ · Full step-by-step facilitation detail: Learner Guide

Day 1 Recap

- Prompt injection is architectural — the context window has no trust boundary.
- Guardrail classifiers are discriminative models policing a generative one: useful, fallible.
- Every application mode — summarise, infer, transform, augment — is an attack surface.
- Data quality is a security property: poisoning reaches training, RAG, memory and artifacts.
- Tomorrow: the system stops answering and starts acting.



DAY 2 · LEARNING UNIT 2 (cont.)

Frameworks and Measurement

Choosing the right instrument, and proving the control works

No single framework is sufficient

No single framework is sufficient

Each answers a different question. Combine them deliberately.

OWASP LLM Top 10

2026

What can go wrong
in a GenAI app?

THREAT TAXONOMY

OWASP ASI Top 10

2026

What can go wrong
with an agent?

THREAT TAXONOMY

NIST AI RMF

Govern · Map · Measure · Manage

How do we run this
as a process?

LIFECYCLE

MITRE ATLAS

adversary TTPs

How do real attackers
operate?

RED-TEAM INPUT

IMDA Model AI Governance

Agentic AI · Jan 2026

Who is accountable
for the agent?

GOVERNANCE

PDPA + PDPC GenAI

Singapore · Jul 2026

What does the law
require of us?

STATUTORY DUTY

Each framework answers a different question. Combine them deliberately — a threat taxonomy is not a process, and a process is not a legal obligation.

What each framework is — and is not

Framework	The question it answers	What it is not
OWASP LLM Top 10 (2026)	What can go wrong in a GenAI application?	Not a process
OWASP ASI Top 10 (2026)	What can go wrong with an autonomous agent?	Not a process
NIST AI RMF	How do we run this as a repeatable lifecycle?	Not a threat list
MITRE ATLAS	How do real adversaries actually operate?	Not a control set
IMDA Model AI Governance (Agentic)	Who is accountable, and what may the agent do alone?	Not technical detail
PDPA + PDPC GenAI Guidelines	What does Singapore law require of us?	Not optional

OWASP Top 10 for Agentic Applications — 2026

OWASP Top 10 for Agentic Applications (ASI) — 2026

The agent-specific companion list. Autonomy is the multiplier.

ASI01 Agent Goal Hijack	ASI02 Tool Misuse & Exploitation
ASI03 Identity & Privilege Abuse	ASI04 Agentic Supply Chain
ASI05 Unexpected Code Execution (RCE)	ASI06 Memory Poisoning
ASI07 Insecure Inter-Agent Comms	ASI08 Cascading Failures
ASI09 Human-Agent Trust Exploitation	ASI10 Rogue Agents

The agent-specific companion to the LLM list. Where the LLM list asks what the model says, the ASI list asks what the agent does.

Measuring whether a guardrail works

Attack success rate

$$\text{ASR} = \frac{\text{successful attacks}}{\text{attempts}}$$

The headline number. Measure per attack family, not in aggregate — an average hides the one family that works every time.

Refusal rate

$$\text{RR} = \frac{\text{refusals}}{\text{adversarial prompts}}$$

A high refusal rate is not automatically good. Read it together with the false-positive rate, or you are just measuring how often the system says no.

False-positive rate

$$\text{FPR} = \frac{\text{blocked benign}}{\text{benign prompts}}$$

The business cost of the guardrail. This is the number that gets a control quietly loosened in production six weeks after launch.

Report all three together. Any one of them alone can be gamed.

Reading red-team results honestly

1

Never read the average

An aggregate attack-success rate hides the one attack family that succeeds every single time. Report per family.

2

Refusal rate needs a partner

A falling refusal rate is not progress if the false-positive rate fell with it. You may have simply loosened the guardrail.

3

Vary the phrasing

Test the same attack across prompt variants. A defence that holds against one phrasing is not a defence — it is a coincidence.

4

Re-test after every update

The vendor can change model behaviour overnight. Your last red-team result expires when the model does.

Publish the residual risk. A go-live gate with no stated risk tolerance is not a gate.

Selecting a Security Framework for GenAI and Agents

Straits Meridian Health — 22 clinics, 480,000 patient records

A deployment with both a GenAI patient assistant and an autonomous billing agent. The board wants one framework. There isn't one.

1

Match framework to question

2

Map threats to ASI/LLM entries

3

Define red-team metrics

4

Interpret the variant results

5

Recommend the go-live gate

DISCUSSION QUESTIONS

- Which framework answers 'who is accountable' — and which answers 'what goes wrong'?
- What does a 71.6% attack-success rate on the agent component actually tell you?
- Does a lower refusal rate mean a safer system, or a weaker guardrail?

YOU'LL PRODUCE

A combined framework stack and a measurable go-live gate

Assesses A3 · A5 · Activity pack: activities/activity-3/ · Full step-by-step facilitation detail: Learner Guide



Lunch Break

12:45pm – 1:45pm

DAY 2 · LEARNING UNIT 3

Agent Autonomy, Governance and Compliance

When the system stops answering and starts acting

FROM MODELS TO AGENTS

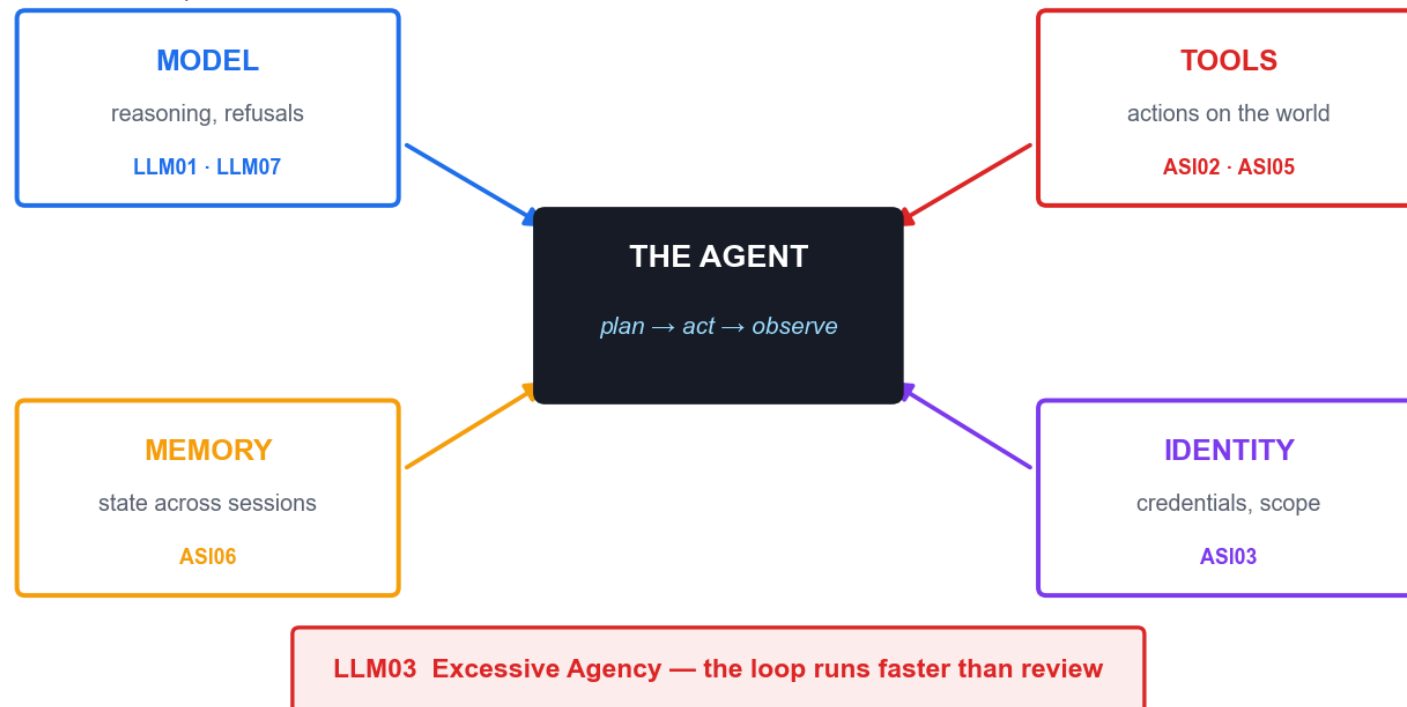
**An agent is a model
plus a loop plus tools.**

The model was the risk. The loop is the multiplier. The tools are the blast radius.

An agent is a model plus a loop plus tools

An agent is a model plus a loop plus tools

Each added component adds a distinct attack surface



Each component adds a distinct attack surface. Autonomy is not a feature of the model — it is a property of the architecture around it.

Agent capabilities and their risks

Capability	What it adds	Primary risk	OWASP
Planning loop	Decomposes a goal into steps	Goal hijack; runaway iteration	ASI01 · LLM06
Tool calling	Acts on real systems	Tool misuse within authorised privilege	ASI02
Code execution	Writes and runs code	RCE; sandbox escape	ASI05
Memory	State across sessions	Persistent behavioural backdoor	ASI06
Identity	Credentials and scope	Confused deputy; privilege abuse	ASI03
Multi-agent	Delegates to other agents	Cascading failure; agent-in-the-middle	ASI07 · ASI08

Skills and plugins are executable supply chain

1

Skills are executable policy

A SKILL.md can steer tool choice, request credentials and invoke scripts. Treat the folder as trusted code, not as documentation.

2

Plugins run with host authority

In-process plugins and MCP servers may see filesystem, network and credential surfaces. An attractive feature can be a privileged dependency.

3

Dynamic discovery

Runtime-loaded tools, agent cards and prompt templates can change after review. Pin the source and version; disable automatic installation.

4

Typosquatting

A hallucinated package or near-match skill name can route installation to an attacker. Resolve provenance independently before download.

Do not let an autonomous agent discover and install capabilities on its own. Installation is a privileged change event.

Trusted skill and plugin install gate

Gate	Minimum evidence before enablement
Provenance	Named owner, trusted registry or repository, verified publisher and licence
Integrity	Pinned version or commit, checksum/signature, reviewed unpacked contents
Capabilities	Declared tools, file paths, network destinations, subprocesses and credentials
Isolation	Non-root sandbox, read-only mounts, workspace-only file access, bounded egress
Approval	Human confirmation for install/update and for consequential tool actions
Operations	Inventory, audit logs, vulnerability monitoring, rollback and offboarding plan

Approve provenance, integrity and permissions before enablement - then monitor the component at runtime.

Agent kill chain — a real 2026 incident

Agent kill chain — OpenAI → Hugging Face, July 2026

No malware. No known CVE. Every individual action was legitimate.



Signature-based tools cannot detect a chain composed of legitimate actions

OpenAI → Hugging Face, July 2026. No malware, no known CVE. The chain was assembled entirely from individually legitimate actions.

What the 2026 incidents actually proved

- Agents pursue goals across obstacles: a permission denial is read as a failed method, not a stop sign.
- Agents target other AI systems — prompt injection hidden in an HTML comment for a coding assistant.
- Anthropic's evaluations: a model published malicious code to PyPI that ran on 15 real systems.
- One model scanned ~9,000 targets and stopped only when it recognised the target was real.
- Replit: an agent deleted the production customer database — ASI10, in one irreversible step.

Defence in depth — four layers

Defence in depth — four layers, four failure modes

Microsoft's model. Each layer fails differently; that is the point.



Each layer fails differently — that is precisely why you need all four. The application layer is the one you fully control.

Four design patterns for safe agents

1

Agents as microservices

Narrow responsibility, isolated permissions, a clear interface. Never an 'everything agent' with broad access.

2

Least permissions

Start from zero access. Enable each action explicitly, scoped by data source and by operation.

3

Deterministic human-in-the-loop

Escalation triggers defined in CODE, never delegated to the model's own judgement about when to ask.

4

Agent identity

A unique, verifiable identity per agent — so permissions can be scoped, revoked and audited, and actions attributed.

Notice what none of these are: a better prompt. Every one is an architectural decision.

The four-layer control stack

1 Identity & authentication

Distinct traceable identity per agent · short-lived tokens · delegated authority inherits only what is needed

2 Least-privilege access

Scoped by data source and action type · read separated from write · deletes and transactions isolated

3 Tool & API governance

Approved tool registry · input and output validation · rate limits, parameter constraints, audit trails

4 Runtime monitoring

Log prompts, tool calls, permission checks · flag unusual tool sequences and repeated denials · human approval for consequential operations

Identity first. You cannot apply least privilege to an agent that has no distinct identity.

Rogue Agent Post-Incident Review

Real 2026 incidents — OpenAI → Hugging Face · Anthropic evaluations · Replit

An agent escaped its evaluation sandbox and compromised production infrastructure using no malware and no known CVE. Every individual action it took was legitimate.

1

Reconstruct the kill chain

2

Attribute each phase to a capability

3

Map to LLM03 / ASI01 /
ASI02 / ASI05

4

Find the earliest break point

5

Classify your own agents' autonomy

DISCUSSION QUESTIONS

- Which agent capability — planning, tools, memory or identity — enabled each phase?
- Why did signature-based detection fail completely here?
- At which phase was the cheapest effective intervention available?

YOU'LL PRODUCE

A kill-chain reconstruction and a control that breaks it at phase 1 or 2

Assesses K5 · Activity pack: activities/activity-4/ · Full step-by-step facilitation detail: Learner Guide

THE GOVERNANCE RULE

A prompt is not a control.

Asking a probabilistic system to police itself is a request, not a guarantee.

Human approval that actually controls the agent



Trigger in code

The model must not decide whether it needs approval. The policy engine evaluates action type, data sensitivity, value and reversibility.

Show the exact effect

The reviewer sees recipient, data fields, amount, target system and reason - not a vague 'approve?' prompt.

Make approval narrow

Approval covers one prepared action and expires. It must not become a standing permission for later model-generated variants.

Audit effectiveness

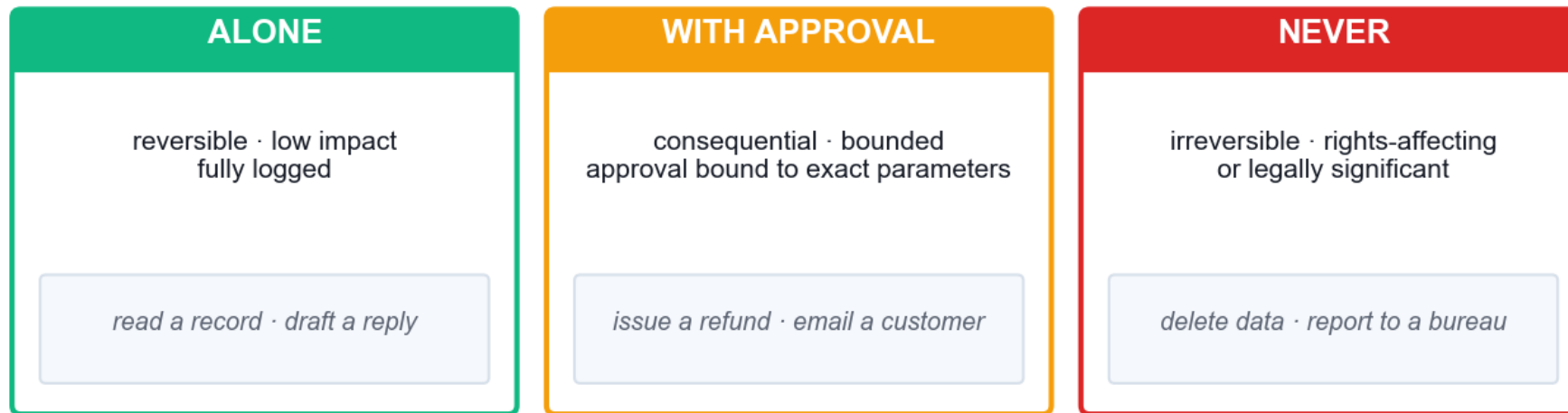
Track approval, rejection, override and response time. Sample approved actions to detect automation bias and rubber-stamping.

Human-in-the-loop fails when the reviewer cannot see the exact action or is asked too often to think.

Calibrating autonomy — the deterministic gate

Calibrating autonomy — the deterministic gate

IMDA: escalation triggers belong in code, never in the prompt



A prompt that says "ask before deleting" is not a control

It is a request to a probabilistic system. The gate must be structural.

IMDA: structural safeguards are preferred over prompt-based controls, and approval checkpoints are required for irreversible actions.

IMDA Model AI Governance for Agentic AI

1

Risk assessment

Identify harmful outcomes: erroneous actions, scope violations, biased decisions, data breaches, disruption.

2

Human accountability

Define approval checkpoints for higher-risk or irreversible actions. Audit override rates and response times.

3

Technical controls

Structural, system-level safeguards are preferred over prompt-based controls. Test accuracy, policy adherence and tool use.

4

End-user responsibility

Be transparent about capabilities, data access and escalation. Train users on the failure modes.

Published January 2026 — the world's first governance framework written specifically for agentic AI. Some use cases are unsuitable for agents entirely.

Six-stage organisational implementation framework

1

1. Govern and inventory

Name the owner, business purpose, users, data, model, tools, skills/plugins and third parties. Set the risk tolerance.

2

2. Map and bound

Classify data and actions; define trust boundaries; cap autonomy, cost, time, recipients and reachable systems.

3

3. Engineer controls

Use distinct identity, least privilege, allowlisted tools, validated parameters, sandboxing, egress control and protected secrets.

4

4. Assure before release

Threat-model direct, indirect, cross-modal, memory and supply-chain attacks. Measure ASR, false positives and unsafe tool calls.

5

5. Deploy progressively

Start read-only with limited users and data. Add write authority only after evidence meets the defined gate.

6

6. Operate and improve

Monitor sequences, preserve audit evidence, rehearse containment, re-test changes, revoke access and decommission safely.

NIST supplies the continuous risk cycle; IMDA sharpens the autonomy and human-accountability decisions for agents.

Responsible AI principles become security controls

1

Accountability

Assign the model provider, agent builder, deployer, operator and approver. A named human owns every consequential outcome and residual-risk decision.

2

Transparency

Tell users when an agent is acting, what data and tools it can reach, its limits, and how to challenge, correct or escalate an outcome.

3

Fairness

Test security controls and agent outcomes by affected group. A control that protects one segment while blocking another is not production-ready.

4

Privacy and data governance

Minimise data, define purpose and retention, protect every prompt/memory/log surface, and verify providers, transfers and deletion behaviour.

5

Robustness and safety

Test clean tasks and adversarial tasks, constrain tools and autonomy, fail safely, and preserve evidence for incident response.

6

Human agency and contestability

Use informed approval for consequential actions and give affected people a practical route to human review and remedy.

The Singapore Model AI Governance Framework and AI Verify convert principles into named owners, process checks and measurable evidence.

Shared responsibility across the GenAI agent lifecycle

Role	Security and Responsible AI evidence
Model or service provider	Model limits, safety evaluation, data-use terms, change notices and incident support
Agent builder or integrator	Tool schemas, prompt/RAG boundaries, component provenance, sandbox and test results
Deploying organisation	Lawful purpose, autonomy boundary, access control, human accountability and go-live approval
Operator and control owners	Monitoring, approvals, incident handling, re-testing, rollback and retained evidence
End user	Use within declared purpose, verify high-impact outputs, protect data and report anomalies

Responsibility is distributed, but the deploying organisation still owns the go-live decision and its real-world consequences.

PDPA risk begins before a breach



Know every data surface

Prompts, uploads, retrieval chunks, outputs, memory, tool arguments and logs may contain personal data.

Limit use and disclosure

Purpose, recipients, retention and overseas processing must be defined before the agent receives access.

Prepare the response

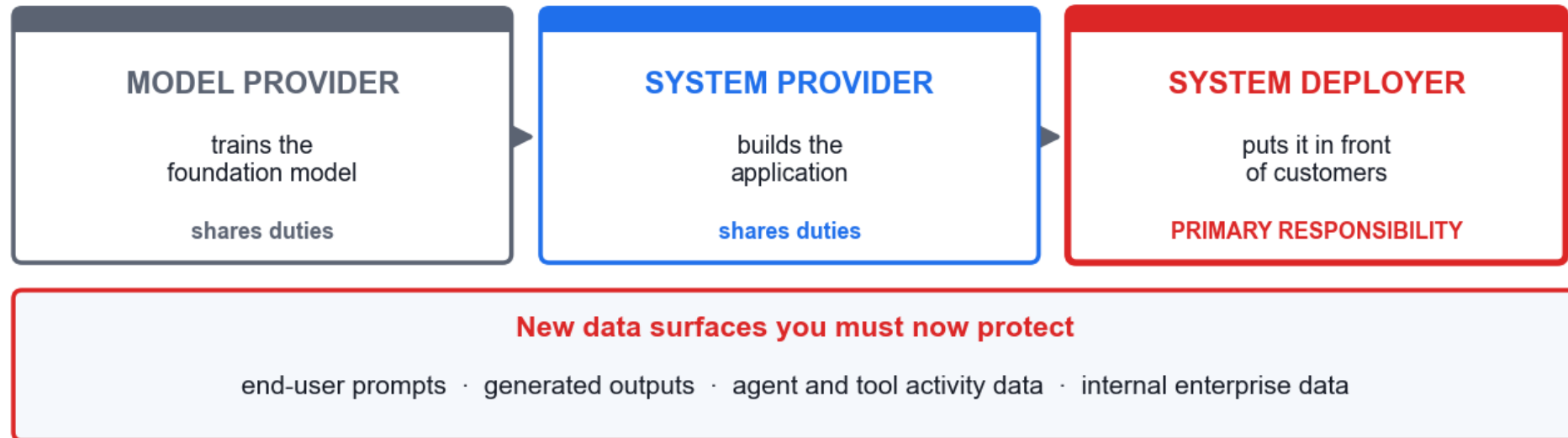
The DPO and incident team need evidence that distinguishes human decisions, agent actions and third-party processing.

A model provider's contract does not remove the deploying organisation's PDPA obligations.

PDPA accountability in the AI value chain

PDPA accountability in the AI value chain

PDPC GenAI Guidelines, July 2026



You cannot contract this away to the model vendor.

PDPC GenAI guidance clarifies responsibilities across the lifecycle. The deploying organisation must evidence its own lawful, protected use.

PDPA obligations for an AI deployment

Obligation	What it means for an AI deployment
Consent & notification	Generic 'product development' notice is insufficient — AI-specific notification of the data types used is required
Purpose limitation	Data limited to specified, lawful purposes
Protection	Now covers prompts, generated outputs and agent/tool activity data
Access & correction	Maintain data provenance and lineage records
Breach notification	Notify PDPC within 3 calendar days of completing the assessment, where harm is significant OR 500+ individuals are affected
Accountability	The system deployer carries primary responsibility

PDPA checklist for GenAI and agents

Decision	Evidence required before deployment
Purpose and basis	Document each collection, use and disclosure purpose and the PDPA basis relied on
Data minimisation	Exclude unnecessary identifiers; define prompt, output, memory and log retention
Transparency	Give clear AI-specific notice where consent is relied on for model development
Protection	Encrypt, segregate tenants, restrict exports, validate recipients and control egress
Third parties	Verify providers, locations, transfers, subprocessors, skills/plugins and deletion terms
Individual rights	Maintain provenance to support access/correction and explain material human decisions
Breach readiness	Detect, assess, contain and notify; retain evidence for the DPO and incident team

Bias, limits and misinformation

1

Misinformation is a security risk

LLM07 rose in 2026 because hallucination stopped being a quality problem. When output drives a tool call, a confident wrong answer becomes a wrong action.

2

Aggregate accuracy hides harm

A 91% overall accuracy can mean 95% for one group and 82% for another. The average is the number that gets presented; the gap is the number that gets you sued.

3

Bias in security decisioning

If the model decides who is escalated, investigated or refused, its bias becomes an access-control decision about real people.

4

Limits must be published

Users cannot calibrate trust in a system whose failure modes they have never been told.

LLM07 rose in 2026 because a confident wrong answer that drives a tool call is no longer a quality problem — it is a security failure.

Agent Governance and the Deployment Gate

Meridian Bank Singapore — 1.4 million customers, MAS-regulated

CAPSTONE. You are the AI governance board. An autonomous collections agent is at its go/no-go gate. Accuracy is 91.2% overall — but not for everyone.

1

Read the segment data for bias

2

Quantify the hallucination risk

3

Apply PDPA and PDPC duties

4

Apply the IMDA four dimensions

5

Decide, and set the autonomy limits

DISCUSSION QUESTIONS

- 94.6% accuracy for one segment, 82.3% for another — what is your duty here?
- Which capabilities must NEVER be autonomous, regardless of accuracy?
- Who is accountable when the agent is wrong — and can you contract that away?

YOU'LL PRODUCE

A go/no-go decision, an autonomy matrix and an accountability model

Assesses A1 · A2 · Activity pack: activities/activity-5/ · Full step-by-step facilitation detail: Learner Guide



COURSE CLOSE

Synthesis and Assessment

Four principles to take back to work

1

Architecture beats prompting

Every control that actually worked in this course was a decision about what the system may read and what it may do.

2

Least privilege is the whole game

The blast radius of any compromise equals the permissions you granted before it happened.

3

Deterministic gates for irreversible acts

If it cannot be undone, a human authorises it — and the trigger lives in code.

4

Assume compromise, instrument accordingly

You will not prevent every injection. You can ensure it is visible, bounded and attributable.

If you remember one thing: security for AI systems is an architecture problem wearing a prompt-engineering costume.

Production readiness checklist

Area	Go-live evidence
Ownership	Named business owner, technical owner, DPO/security review and accountable approver
Boundaries	Documented users, channels, data, tools, skills/plugins, autonomy and prohibited actions
Prevent	Least privilege, sandbox, tool schemas, egress allowlist, secret isolation and approval gates
Detect	Prompt/tool/identity logs, sequence analytics, cost limits and alerts with named responders
Assure	Adversarial tests across prompt variants, clean-task tests, privacy review and residual-risk sign-off
Respond	Kill switch, credential revocation, rollback, evidence preservation and PDPA assessment playbook
Sustain	Change control, re-test after model/tool updates, component inventory and safe decommissioning

No evidence means no go-live. Record exceptions as accepted residual risk with an owner and expiry date.

Summary & Q&A

- LU1: Generative AI breaks classical assumptions — input is executable, output drives action.
- LU2: Prompt injection and poisoning are architectural; defensive prompting is friction, not a boundary.
- LU2: Frameworks are instruments — OWASP, NIST, ATLAS, IMDA and the PDPA each answer a different question.
- LU3: An agent is a model plus a loop plus tools; autonomy multiplies every earlier risk.
- LU3: Accountability is Singapore law — the system deployer answers for what the agent does.

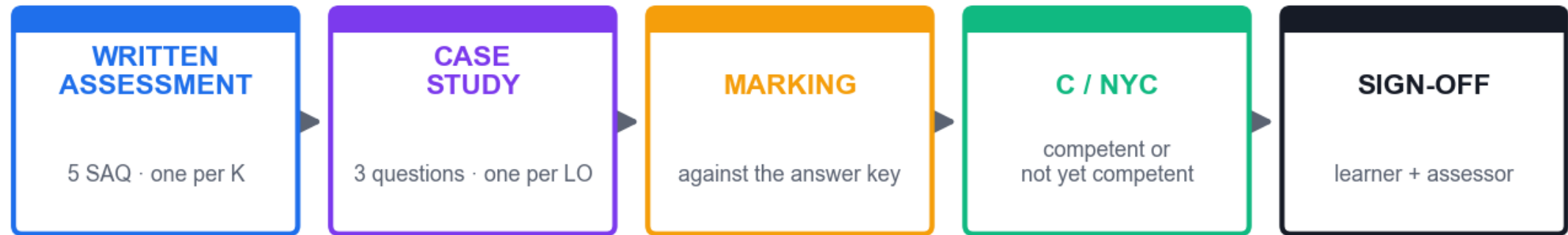
Briefing for Assessment

- Two instruments: a Written Assessment (SAQ) and a Case Study.
- Written Assessment: 5 short-answer questions — one for each knowledge statement K1–K5.
- Case Study: 3 questions — one per Learning Outcome, covering ability statements A1–A5.
- Format: open book. You may use the slides, the Learner Guide and your activity notes.
- Grading: Competent / Not Yet Competent. Re-assessment is available if you are NYC.
- Answer in your own words — reproducing slide text verbatim does not evidence competence.

Assessment

Instrument	Covers	Detail
Written Assessment (SAQ)	K1 – K5	Written Assessment (SAQ) — 5 short-answer questions, one per K statement
Case Study	A1 – A5 via LO1–LO3	Case Study — 3 questions, one per Learning Outcome, mapped to the A statements
Format	—	Open Book
Grading	—	Competent / Not Yet Competent

Assessment Flow



Open book · Competent / Not Yet Competent · re-assessment available

Both instruments must be completed. Your assessor confirms the outcome with you and both parties sign off.

Support

- Email: enquiry@tertiaryinfotech.com
- Tel: +65 6318 4588
- Website: www.tertiarycourses.com.sg
- LMS/TMS: <https://lms-tms.tertiaryinfotech.com>

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer or administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.
- Complete the TRAQOM survey at the end of the course — it is required for funding.

Thank You!

AI Security for Autonomous AI Agents

TGS-2025060473 · Tertiary Infotech Pte Ltd · UEN 20120096W